

ONLINE RETAIL SALES DATABASE

Internship Project – Elevate Labs | Project Report

1. Introduction

This project designs and implements a relational MySQL database for an online retail/e-commerce platform. The database stores customer, product, order, order-item and payment information and supports sales analysis and reporting.

2. Abstract

A normalized database structure was created to reduce data redundancy and maintain data integrity. Five related tables—Customers, Products, Orders, Order_Items and Payments—were implemented with primary keys, foreign keys, NOT NULL and UNIQUE constraints. Sample retail data was inserted and SQL queries using JOIN, GROUP BY, aggregate functions and ORDER BY were executed for reporting and validation.

3. Tools Used

Tool	Purpose
MySQL Server 8.0	Database engine and SQL execution
MySQL Workbench	Database creation, table design, data insertion and query testing
dbdiagram.io	Entity-Relationship (ER) diagram
GitHub	Project repository and versioned project files

4. Steps / Methodology

- **Database setup:** Created the **online_retail_db** database in MySQL.
- **Schema design:** Created Customers, Products, Orders, Order_Items and Payments tables.
- **Keys and constraints:** Applied primary keys, foreign keys, AUTO_INCREMENT, NOT NULL and UNIQUE constraints.
- **Normalization:** Separated order-level and product-level details using Order_Items to support a normalized relational design and reduce repeating data.
- **Sample data:** Inserted 5 customers, 8 products, 8 orders, 13 order-item records and 8 payment records.
- **Analysis:** Executed queries for daily order totals, category sales, product quantities sold, payment-method summaries and other reporting tasks.
- **ER modelling:** Created the ER diagram in dbdiagram.io and exported it as PNG.
- **Documentation:** Added the SQL script, ER diagram and project description to the GitHub repository.

5. Database Structure

Table	Primary Key	Main Relationships
Customers	customer_id	Referenced by Orders
Products	product_id	Referenced by Order_Items
Orders	order_id	customer_id → Customers
Order_Items	order_item_id	order_id → Orders; product_id → Products
Payments	payment_id	order_id → Orders

6. 3NF / Normalization

The design separates independent entities into individual tables. Customer attributes are stored in Customers, product attributes in Products, order-level information in Orders, and the products belonging to each order in Order_Items. Payment information is maintained separately in Payments. This avoids repeating customer and product details across multiple orders and supports a 3NF-oriented relational structure.

7. SQL Reporting and Validation

The project includes SQL queries using SELECT, JOIN, GROUP BY, SUM, COUNT and ORDER BY. Examples include daily order summaries, sales by product category, total quantity sold per product, and payment totals grouped by payment method and status. Order-item values can also be compared with stored order totals using $\text{quantity} \times \text{unit_price}$ for data validation.

8. Project Repository

GitHub repository: github.com/sufiyanabu92814-svg/online-retail-sales-database

9. Conclusion

The project demonstrates the design and implementation of a normalized relational database for an online retail system. It combines schema design, keys and constraints, sample data, SQL analysis, validation, ER modelling and documentation. The resulting database provides a structured foundation for storing retail transactions and generating useful sales reports.